

Viacheslav Androsov

Frontend Developer (React / Vue / TypeScript)

Vietnam | Open to remote work

+7 993 075 84 04 · bandrone.man@gmail.com · t.me/awaitMeBro · [portfolio](#) · github.com/Homodeus7 · linkedin.com/in/bandrone

SUMMARY

Frontend developer with 4+ years of commercial experience. Started with outsourced Web3 interfaces, then moved to product teams: a B2B/FinTech platform and an enterprise low-code tool. My main specialisation is complex SPAs on React and Vue 3 with TypeScript: I design the frontend architecture (FSD) and put legacy code back in order. I work with AI tools daily (Claude Code, agentic pipelines): prototypes, tests, reviews, documentation — I ship features faster without losing quality.

CORE SKILLS

Frontend: TypeScript, React, React Native (Expo), Vue 3 (Composition API), Next.js, Redux Toolkit, Zustand, Pinia, TanStack Query / Table, React Hook Form, VueUse

UI / Styling: Tailwind CSS, shadcn/ui (Radix), SCSS, Storybook, responsive layouts

Quality / Tooling: Vite, Zod / Yup, OpenAPI, MSW, Vitest, Jest, React Testing Library, Playwright, FSD, CI/CD, Git

Web3: ethers.js, web3.js, MetaMask, WalletConnect, smart-contract integration

AI: AI-assisted development — Claude Code, agentic pipelines, MCP

WORK EXPERIENCE

NDA — Frontend Developer (Vue)

Feb 2026 — present

Enterprise low-code platform: a visual business-process editor that turns a diagram into an executable process

- Develop the core of the product — the editor, where the user assembles a process diagram out of blocks with the mouse and the platform turns it into an executable pipeline.
- Designed undo and redo for the whole editor — 40+ operation types, including moving blocks between diagrams (pools, nested subprocesses). User actions collect in a buffer and are compacted before being sent: the backend receives the result, not every mouse movement.
- Sped the editor up from ~25 to a stable 60 FPS on diagrams of 100+ blocks: removed the idle repaint loop, took scene geometry out from under Vue reactivity and rebuilt the frame so that measuring and painting no longer interleave.
- Moved block contents onto Vue components over the canvas: forms inside blocks are assembled from a description supplied by the backend, so a new block type plugs in with no hand-written markup on the frontend.
- Integrated the editor's AI assistant into the frontend: the dialogue, a preview of the suggested steps on the diagram, applying them through the shared action buffer. In parallel, set up how the team works with Claude Code — test generation, change review, keeping architecture documentation alive.

Stack: Vue 3 (Composition API), TypeScript, Pinia, Vite, WebSocket / STOMP, Keycloak, Tailwind CSS, Zod, Vitest, VueUse.

NDA — Frontend Developer (React)

Apr 2025 — Jan 2026

FinTech payment-gateway platform: merchant portal (B2B), admin panel and an embeddable payment form

- Advanced the frontend architecture along FSD: three applications grew to 50+ modules in one codebase and still do not get in each other's way — every team has its own boundaries.
- Built the merchant portal sections: API integrations, webhooks, balances. Instead of every widget polling for data on its own, set up a shared query cache — load on billing dropped by 30% (measured on server metrics).

- Made the analytics dashboards and transaction tables: filters work without a round trip to the server, a status update repaints one row rather than the whole table — 500+ transactions scroll smoothly.
- Introduced dynamic validation for financial forms: the set of fields depends on the payout method and the currency — fewer input errors and support tickets.
- Connected error monitoring in Sentry tied back to the source code, and took part in building the embeddable payment form with QR payments.

Stack: *React, TypeScript, Redux Toolkit, TanStack Query / Table, React Hook Form, shadcn/ui (Radix), Tailwind CSS, Sentry, Vite, FSD.*

MGS LLC — Frontend Developer (React, React Native)

Mar 2024 — Apr 2025

Residential-complex management ecosystem: a web dashboard for management companies and a mobile app for residents

- Built from scratch a mobile app for residents and management-company staff on React Native: requests, payments, documents, push notifications.
- Took part in moving the legacy web app onto FSD architecture without pausing development. New developers started carrying their first task through to release in days instead of weeks.
- Set up generation of a typed API client from the OpenAPI schema: types stopped drifting from the backend and manual synchronisation disappeared entirely.
- Sped the web app up: split the bundle by routes, added request caching and removed extra re-renders — the initial load and page transitions became noticeably faster.
- Built a permission system on CASL: role rules live in one place and components simply ask "is this action allowed" — a new role is added by editing one file rather than by checks all over the code.
- Set up API mocks on MSW — the frontend stopped waiting for the backend to be ready, and the same mocks were reused in tests.

Stack: *React, React Native (Expo), TypeScript, Zustand, TanStack Query, React Hook Form + Yup, CASL, OpenAPI / Orval, HeroUI, Tailwind CSS / NativeWind, MSW, Vite.*

NDA (outsourcing studio) — Frontend Developer

Jun 2022 — Feb 2024

Custom interface development for Web3 applications, DeFi and GameFi startups

- Built interfaces for Web3 products on Vue 3 and TypeScript.
- Connected MetaMask and WalletConnect crypto wallets through ethers.js: sign-in by wallet, transaction signing, network switching and clear error handling.
- Showed blockchain data in real time — prices, liquidity, yield — over WebSocket, with no page reloads.
- Took part in building an NFT platform with a revenue-sharing model (Stake / Trade / Earn).
- Built promo landing pages with 3D graphics and animations (three.js, GSAP) to bring users into the clients' products.
- Worked through UX together with the product manager and the designer: loading states, error handling, onboarding for new users.

Stack: *Vue 3 (Composition API), TypeScript, Pinia / Vuex, TanStack Query, vee-validate + Yup, Vite, SCSS / Tailwind CSS, ethers.js, web3.js, MetaMask, WalletConnect, ApexCharts, GSAP, three.js.*

PET PROJECTS

FoodIQ — pet project

foodiq.space

Calorie-tracking web app: the user describes a meal in plain text and AI works out the calories and macros

- Frontend on Next.js 16 and backend on NestJS: from the idea and the design through to a production deployment.
- Built AI meal parsing from free text: products, weights, calories and macros are worked out automatically, with a confidence score to check them against.
- Built the food diary, goals and plans, custom products and recipes over a public database, statistics with charts, the marketing landing page and the admin panel.
- Made token-based billing: a free monthly allowance, one-off packs and a subscription, paid in crypto through NOWPayments.

- Set up the infrastructure: a typed API client from OpenAPI (Orval), FSD architecture, Vitest and Playwright tests, Storybook, dark theme, i18n (ru/en).

Stack: Next.js 16 (App Router), React 19, TypeScript, Tailwind CSS v4, shadcn/ui (Radix), TanStack Query, Zustand, React Hook Form + Zod, OpenAPI, Recharts, Vitest, Playwright, Storybook, FSD.

androsov.space — portfolio

[portfolio](#)

Portfolio site where every claim that carries a number is a demo you can run on the same page

- Next.js 16 (App Router), React 19 and TypeScript in two locales: the design, the copy, the code and the deployment are all mine.
- Eight interactive demos, each rebuilding one feature from a production project: the undo/redo buffer, the canvas frame budget, a transaction table, a dynamic payout form, permissions on CASL, wallet states. The logic sits in separate entities and is covered by unit tests.
- Two of the demos are written in Vue and run as islands inside the React app: Vue arrives in a chunk of its own once the island reaches the screen, and never enters the initial payload.
- FSD boundaries are held by two hand-written ESLint rules (an import going upwards through the layers, an import bypassing a slice's public API), and case content is checked by Zod at import time: a half-translated case fails the build rather than shipping.
- Covered the demos with Playwright e2e tests, including an accessibility pass through axe.

Stack: Next.js 16 (App Router), React 19, TypeScript, Tailwind CSS v4, GSAP, Lenis, next-intl, Zod, Vue 3, CASL, Vitest, Playwright, FSD, Vercel.